

Genesis v1 Local Setup and End-to-End Run Guide

Audience: one operator running Genesis locally on macOS or Linux, using the current **codex/v1-pass2-integration** branch as the best pre-setup hardening baseline.

Goal: set up a serious local environment, install and prove the heavy backends, run one real project from a project spec, and review the resulting paper artifacts for actual writing and figure quality.

Current best branch: **codex/v1-pass2-integration**. Do not use the stale dirty local **main** checkout for the proving run.

Websites you will likely use during setup:

- Ollama download: <https://ollama.com/download>
- Ollama model library: <https://ollama.com/library>
- Gemma overview and official docs: <https://ai.google.dev/gemma>
- Groq console and API keys: <https://console.groq.com/keys>
- Hugging Face access tokens: <https://huggingface.co/settings/tokens>
- Sentence-transformer model page: <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>
- Semantic Scholar API: <https://www.semanticscholar.org/product/api>
- PyTorch local install guide: <https://pytorch.org/get-started/locally/>
- PyTorch Geometric install docs: <https://pytorch-geometric.readthedocs.io/en/latest/install/installation.html>
- Chroma docs: <https://docs.trychroma.com/>
- MacTeX / BasicTeX for pdflatex: <https://www.tug.org/mactex/>

1. Choose the Right Codebase State

Use **codex/v1-pass2-integration**. It includes the strongest harness policy, optional heavy backend support paths, and the latest verification/publication hardening.

```
git fetch origin
git checkout codex/v1-pass2-integration
# or create a clean proof checkout
# git worktree add ../genesis-local-proof origin/codex/v1-pass2-integration
```

Verify the branch before doing anything else:

```
git rev-parse --abbrev-ref HEAD
git log --oneline -3
```

2. Recommended Local Environment

2.1 Python Version

Use Python 3.11 if possible. It is the safest target for the local ML stack: torch, sentence-transformers, chromadb, and gpytorch.

```
conda create -n genesis-v1 python=3.11 -y
conda activate genesis-v1
python --version
```

2.2 Core Install

From the v1 directory, install the full extras set rather than the minimal quick-start path.

```
cd v1
python -m pip install --upgrade pip setuptools wheel
python -m pip install -e ".[dev,data,verification,paper,ml,domain]"
```

Verify the install at a high level:

```
python - <<'PY'
mods = [
    'click','requests','pydantic','rich','loguru',
    'sqlalchemy','alembic','chromadb',
    'torch','sentence_transformers','torch_geometric',
    'gpytorch','sklearn','bibtexparser','jinja2','pylatex','matplotlib','astropy'
]
for m in mods:
    try:
        __import__(m)
        print(m, 'OK')
    except Exception as e:
        print(m, 'MISSING', e)
PY
```

2.3 If PyTorch or PyTorch Geometric Fails

- Install torch first from the official selector at <https://pytorch.org/get-started/locally/>.
- Install torch-geometric using the exact version matrix at <https://pytorch-geometric.readthedocs.io/en/latest/install/installation.html>.
- Then rerun the import check.

3. System Tools You Need

3.1 pdflatex

Genesis writes LaTeX and tries to compile it. Without pdflatex, it falls back to a minimal PDF. For a real proof run, install a TeX distribution.

- macOS full install: MacTeX from <https://www.tug.org/mactex/>
- macOS smaller install: BasicTeX from the same site if you are okay installing missing packages later.
- Linux: TeX Live or your distro equivalent.

```
which pdflatex
pdflatex --version
```

3.2 Ollama

The checked-in runtime config is local-first and defaults to Ollama. Install it from <https://ollama.com/download> and make sure the daemon is running.

```
ollama --version
ollama ps
```

3.3 Local Models

Current checked-in local models:

- primary: qwen2.5-coder:14b
- fallback 1: llama3.3:70b

- fallback 2: llama3.2:3b

```
ollama pull qwen2.5-coder:14b
ollama pull llama3.2:3b
# optional if your machine can host it
ollama pull llama3.3:70b
```

If you want Gemma locally, use <https://ai.google.dev/gemma> and a matching Ollama Gemma model such as <https://ollama.com/library/gemma3>. Gemma is optional; the current Genesis config does not default to it.

4. Accounts, Keys, and Data Sources

For a serious local run, create these even if you stay mostly local:

- Semantic Scholar API key: <https://www.semanticscholar.org/product/api>
- Groq API key: <https://console.groq.com/keys>
- Hugging Face token: <https://huggingface.co/settings/tokens>

Suggested .env in the v1 repo root or your home directory:

```
SEMANTIC_SCHOLAR_API_KEY=your_semantic_scholar_key
GROQ_API_KEY=your_groq_key
OLLAMA_BASE_URL=http://127.0.0.1:11434
GENESIS_CACHE_ROOT=/absolute/path/to/local/genesis_cache
```

5. Prove the Heavy Backends

5.1 Taste Store and GPyTorch Path

```
cd v1
python scripts/init_taste.py
python - <<'PY'
from genesis.taste.gp_model import TasteGP
print('taste backend:', TasteGP().backend)
PY
```

If gpytorch is installed, you want this to say gpytorch. If it says scipy, the fallback path is still active.

5.2 Ledger Backend

```
python - <<'PY'
from pathlib import Path
from genesis.storage.ledger import ExperimentLedger
ledger = ExperimentLedger(Path('tmp_ledger.sqlite3'))
print('ledger backend:', ledger.backend)
PY
```

5.3 Manifold Backend

```
python - <<'PY'
from pathlib import Path
from genesis.storage.manifold import ManifoldIndex
manifold = ManifoldIndex(Path('tmp_manifold'))
print('manifold backend:', manifold.backend)
PY
```

5.4 Embedding Path

```
python - <<'PY'
from genesis.taste.features import ExperimentFeatureExtractor
from genesis.models import ExperimentProposal
```

```

extractor = ExperimentFeatureExtractor()
proposal = ExperimentProposal(
    description='warmup schedule experiment',
    code_diff='warmup_ratio=0.2',
    expected_metric=0.7,
    expected_trajectory=[0.2, 0.4, 0.7],
    compute_budget='local_gpu',
    model_parameter_count=1000,
)
vec = extractor.extract(proposal)
print('feature length:', len(vec))
print('first 8 dims:', vec[:8])
PY

```

6. Build the Local Research State

6.1 Initialize Taste

```

cd v1
python scripts/init_taste.py

```

6.2 Build a Manifold

For the first real proof run, use an ML-efficiency project rather than astrophysics. It is the lowest-friction local path.

Provide your own small JSON corpus of papers. Genesis can fall back to a tiny built-in corpus, but that is not a meaningful proof run.

- Manually curate 20 to 100 papers from Semantic Scholar and save them as JSON.
- Use Semantic Scholar search results and normalize them into the script's expected shape.
- Optionally pull paper metadata from Hugging Face datasets or a local export if you already have one.

Minimum expected paper fields:

- paper_id
- title
- abstract
- year
- authors
- citation_count
- domain
- citations
- url

```

cd v1
python scripts/build_manifold.py --domain ml_efficiency --input /absolute/path/to/ml_efficiency_papers.json

```

7. Provider Configuration Choices

- Pure local: Ollama only. Cleanest first proof.
- Hybrid local plus remote fallback: Ollama primary, Groq fallback.

- Optional Gemma route: pull a Gemma model locally and change the model mapping in a local working copy of .opencode/oh-my-openagent.jsonc.

8. Create a Real Project Spec

Use a small but real project. Recommended first proving spec:

```
{
  "research_question": "Investigate the effect of learning rate warmup on convergence speed and training stability",
  "domain": "ml_efficiency",
  "compute_budget": "local_gpu",
  "time_budget_hours": 3,
  "domain_knowledge_model": "none",
  "output_dir": "/absolute/path/to/projects",
  "success_criteria": [
    "Produce at least one verification-passing run with a clear primary metric.",
    "Generate a final LaTeX paper draft with figures and references.",
    "Ground major claims in citations or explicit synthetic benchmark evidence."
  ],
  "oracle_hints": [
    "metric consistency",
    "sample consistency"
  ]
}
```

9. Run One Real End-to-End Project

9.1 Start the Runtime Services

```
ollama serve
# in another terminal
ollama ps
```

9.2 Run Genesis

```
cd v1
python -m genesis.cli.main run --project-id warmup-proof-01 --spec /absolute/path/to/specs/warmup_mlp_spec.json
```

Use max-runs 5 for the first proof because the harness now follows a 3 adversarial plus 2 escalation model.

9.3 Live Monitoring

```
python -m genesis.cli.main status --project-id warmup-proof-01 --root /absolute/path/to/projects
python -m genesis.cli.main results --project-id warmup-proof-01 --root /absolute/path/to/projects
```

10. Check the Outputs and Paper Quality

10.1 Required Artifacts

- spec.json
- decomposition.json
- adversarial_criteria.json
- knowledge/oracle.py
- knowledge/taste_snapshot.json
- runs/instruction.md
- runs/trace.json

- runs/result.json
- runs/adversarial_report.json
- runs/verification_report.json
- outputs/paper/main.tex
- outputs/paper/main.pdf
- outputs/paper/synthesis_report.json
- outputs/paper/run_index.json

10.2 Paper Quality Checklist

- Abstract states the actual task, method, and observed outcome clearly.
- Introduction is specific rather than generic filler.
- Methods match the run artifacts.
- Results cite real metrics from the run.
- Discussion acknowledges limitations or failed verification if present.
- Figures are readable, labeled, and relevant.
- Captions explain what the figure shows.
- No placeholders, malformed citations, or repeated generic phrases.
- No compile fallback artifacts if you expected real pdflatex output.

10.3 Files To Inspect

```
open /absolute/path/to/projects/warmup-proof-01/outputs/paper/main.pdf
less /absolute/path/to/projects/warmup-proof-01/outputs/paper/main.tex
cat /absolute/path/to/projects/warmup-proof-01/outputs/paper/synthesis_report.json
cat /absolute/path/to/projects/warmup-proof-01/outputs/paper/run_index.json
cat /absolute/path/to/projects/warmup-proof-01/outputs/paper/citation_flags.json
find /absolute/path/to/projects/warmup-proof-01/outputs/paper/figures -name '*.metadata.json' -print
```

11. Acceptance Standard for the First Real Local Proof

- Heavy backend checks succeed, or you explicitly document which fallback path remained active.
- The manifold build succeeds with your own paper corpus.
- Genesis runs from a real spec without mocked runtime shortcuts.
- A final PDF and LaTeX source are produced.
- The paper contains at least one useful figure with metadata.
- Verification and adversarial reports exist for each run.
- You can explain from the artifacts why the system succeeded or halted.

Appendix: Copy-Paste Command Sequence

```
# 1. get the best branch
git fetch origin
git checkout codex/v1-pass2-integration

# 2. create env
conda create -n genesis-v1 python=3.11 -y
conda activate genesis-v1

# 3. install deps
cd v1
python -m pip install --upgrade pip setuptools wheel
python -m pip install -e ".[dev,data,verification,paper,ml,domain]"

# 4. install system tools separately
# - pdflatex via MacTeX / BasicTeX
# - Ollama from ollama.com/download

# 5. pull local models
ollama pull qwen2.5-coder:14b
ollama pull llama3.2:3b

# 6. set environment
export OLLAMA_BASE_URL=http://127.0.0.1:11434
export SEMANTIC_SCHOLAR_API_KEY=...
export GROQ_API_KEY=...
export GENESIS_CACHE_ROOT=$HOME/genesis_cache

# 7. prove heavy backends
python scripts/init_taste.py
python - <<'PY'
from pathlib import Path
from genesis.storage.ledger import ExperimentLedger
from genesis.storage.manifold import ManifoldIndex
from genesis.taste.gp_model import TasteGP
print('ledger backend:', ExperimentLedger(Path('tmp_ledger.sqlite3')).backend)
print('manifold backend:', ManifoldIndex(Path('tmp_manifold')).backend)
print('taste backend:', TasteGP().backend)
PY

# 8. build manifold
python scripts/build_manifold.py --domain ml_efficiency --input /absolute/path/to/ml_efficiency_papers.json

# 9. run project
python -m genesis.cli.main run --project-id warmup-proof-01 --spec /absolute/path/to/specs/warmup_mlp_spec.json

# 10. inspect outputs
python -m genesis.cli.main status --project-id warmup-proof-01 --root /absolute/path/to/projects
python -m genesis.cli.main results --project-id warmup-proof-01 --root /absolute/path/to/projects
```